

Vận Hành 3 Môi Trường Khác Nhau Dev, UAT và Prod

1. Tổng quan:

1.1. Tổng quan về vận hành 3 môi trường:

Trong quy trình phát triển phần mềm, một ứng dụng bắt buộc phải trải qua nhiều môi trường vận hành trước khi tới tay người dùng cuối:

- **Dev (Development):** Cần sự linh hoạt, bật hot-reload, map port trực tiếp để lập trình viên test API và debug.
- **UAT (User Acceptance Testing):** Môi trường kiểm thử giả lập hệ thống thật với cơ sở dữ liệu độc lập.
- **Prod (Production):** Môi trường thực tế, đặt nặng yếu tố an toàn - bảo mật - hiệu năng (đóng port kín, giới hạn CPU/RAM, bật chính sách tự khởi động lại).

Thách thức đặt ra là làm sao cấu hình hệ thống đồng bộ cho cả 3 môi trường nhưng vẫn đảm bảo tính cách ly và bảo mật riêng biệt.

1.2. Phương pháp và lý do lựa chọn:

Thay vì viết 3 file Docker Compose độc lập gây lặp lại cấu hình, báo cáo này áp dụng kỹ thuật **Kế thừa & Ghi đè (Multiple Compose Files)** kết hợp **Cách ly biến môi trường (.env)** vì những lợi ích sau:

1. **Tối ưu mã nguồn (Nguyên lý DRY):** Tách toàn bộ cấu hình chung (tên dịch vụ, network nội bộ, volume) vào file base docker-compose.yml, loại bỏ việc sao chép lặp lại code.
2. **Linh hoạt và Dễ bảo trì:** Các file môi trường (docker-compose.dev.yml, docker-compose.prod.yml) chỉ cần viết ngắn gọn để ghi đè các thông số đặc thù. Khi thêm dịch vụ mới, chỉ cần sửa một nơi tại file base.
3. **Bảo mật dữ liệu nhạy cảm:** Tách biệt "khung xương" hệ thống (file YAML) khỏi "thông tin thực tế" (mật khẩu, API key) qua các file .env riêng biệt. Các file cấu hình có thể push lên GitHub, còn tệp chứa mật khẩu thật (.env.prod) sẽ được giữ bảo mật trên server.

4. **Hỗ trợ tự động hóa (CI/CD):** Cú pháp gọi chuỗi file -f và --env-file giúp các công cụ tự động (GitHub Actions, GitLab CI) dễ dàng nhận diện môi trường để deploy tự động.

2. Kỹ thuật kế thừa và ghi đè cấu hình của Docker Compose:

Thay vì viết 3 file độc lập gây lộn xộn, ta có thể áp dụng kỹ thuật **kế thừa và ghi đè cấu hình (Override)** của Docker Compose bằng cách viết một file compose cơ sở (base) chứa thông tin chung và đối với mỗi môi trường sẽ có một file chứa thông tin riêng mỗi môi trường để override:

1. **docker-compose.yml (file base):** Định nghĩa khung xương hệ thống, tên container, volume, network và các biến template.
2. **docker-compose.dev.yml (file override môi trường dev):** Map port trực tiếp ra ngoài để tiện test, bật hot-reload, ...
3. **docker-compose.uat.yml (file override môi trường uat):** Cấu hình tương tự Prod nhưng sử dụng cơ sở dữ liệu test riêng biệt.
4. **docker-compose.prod.yml (file override môi trường prod):** Bỏ map port trực tiếp, giới hạn RAM/CPU, bật restart policy, cấu hình proxy bảo mật, ...

Khi tiến hành triển khai ứng dụng, tùy thuộc vào máy chủ hiện tại thuộc môi trường nào, quản trị viên sử dụng cú pháp kết hợp file tương ứng, ví dụ môi trường dev:

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d
```

***Lưu ý:** Khi gọi nhiều file -f liên tiếp, Docker Compose sẽ ưu tiên đọc các file đứng sau để ghi đè (override) các giá trị trùng lặp của các file đứng trước. File docker-compose.yml (base) luôn phải được gọi đầu tiên làm nền móng.

3. Quản lý biến môi trường (.env) cách ly:

Không nên thiết lập cứng các giá trị của các biến môi trường trong file docker-compose.yml mà phải tách giá trị ra một file .env để không bị xung đột giữa các môi trường vì có thể mỗi môi trường sử dụng giá trị khác nhau và đảm bảo một số giá trị nhạy cảm không bị lộ ra ngoài vì file .env sẽ không commit lên github, ta chỉ commit file .env.example để khai báo các biến cần sử dụng mà không điền giá trị. Và mỗi môi trường sẽ có một file .env tương ứng:

1. **.env.example:** File mẫu chứa danh sách tên các biến cần thiết cho dự án nhưng không bao gồm giá trị cụ thể (hoặc chỉ để giá trị mặc định/giả định). File này được phép commit lên GitHub để các thành viên khác trong đội ngũ biết dự án cần những cấu hình nào để khởi chạy.
2. **.env.dev:** File chứa các giá trị cấu hình cụ thể phục vụ cho môi trường phát triển (Development) cục bộ của lập trình viên (ví dụ: kết nối tới database local, bật chế độ debug, port chạy thử nghiệm). File này thường được giữ dưới máy cá nhân và không commit lên Git.
3. **.env.prod hoặc .env.uat:** File chứa các thông số cấu hình thực tế, bảo mật cao dành riêng cho môi trường vận hành (Production) như: mật khẩu database thật, API key của bên thứ ba, chúng chỉ bảo mật. File này tuyệt đối không được commit lên GitHub mà sẽ được cấu hình trực tiếp trên server hoặc thông qua các công cụ CI/CD.

Khi tiến hành triển khai ứng dụng, tùy thuộc vào máy chủ hiện tại thuộc môi trường nào, quản trị viên sử dụng cú pháp kết hợp file tương ứng, ví dụ môi trường dev thì lập trình viên copy file .env.example thành file .env.dev, điền các thông số local rồi chạy lệnh:

```
docker compose --env-file .env.dev up -d
```

Mặc định Docker Compose sẽ đọc file .env nằm cùng thư mục với file docker-compose.yml, lệnh trên sẽ chỉ định file .env.dev để đọc và truyền giá trị biến.

4. Use case từ demo đã làm:

- 4.1. Chuẩn bị mã nguồn và các file cấu hình:
- 4.2. Cấu hình môi trường Phát triển
- 4.3. Khởi dựng hệ thống và Kiểm tra kết quả tại Local
- 4.4. Cấu hình và Mô phỏng triển khai môi trường Thực tế (Production)

5. Sơ đồ luồng hoạt động:

